

Cursul 10
Shell scripting

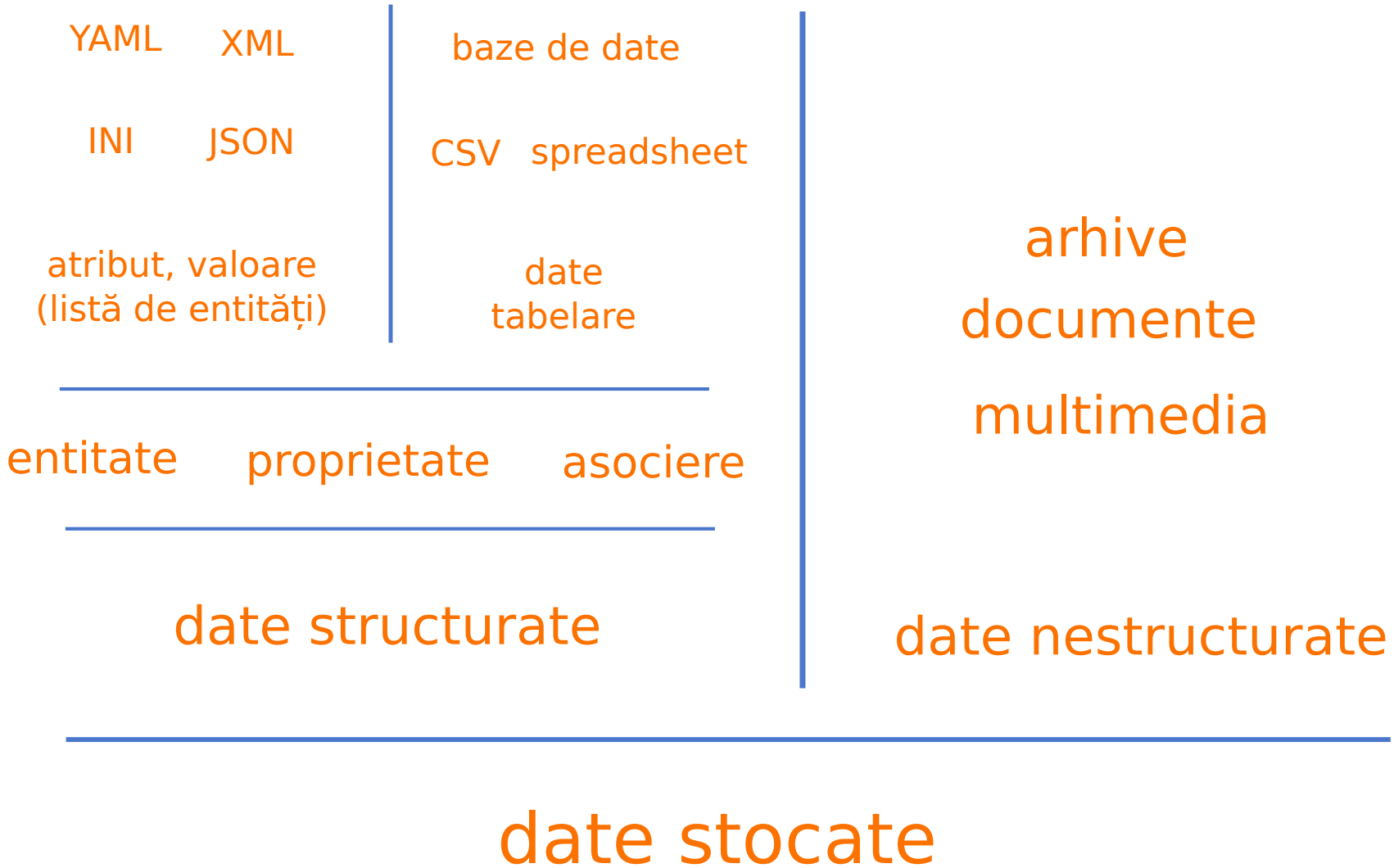
Utilizarea Sistemelor de Operare (USO)
27 noiembrie 2017

Departamentul de Calculatoare

Moto

Some people, when confronted with a problem, think "I know, I'll use regular expressions." Now they have two problems.

Jamie Zawinski (JWZ)


YAML
XML
INI
JSON
atribut, valoare
(listă de entități)
baze de date
CSV
spreadsheet
date
tabelare
entitate
proprietate
asociere
date structurate
arhive
documente
multimedia
date nestructurate
date stocate



http://www.quickmeme.com/p/3vzsz0

- Suport (Introducere în sisteme de operare)
  - Capitolul 12 – Shell scripting
    - Secțiunile 12.4, 12.5, 12.6, 12.9
- Regular Expressions Cookbook
  - Capitolul 1 – Introduction to Regular Expressions
  - Capitolul 2 – Basic Regular Expression Skills

În această săptămână la USO

- fără laboratoare de USO în perioada miercuri-vineri, 29 noiembrie - 1 decembrie 2017
- tema 3 are termen miercuri, 29 noiembrie 2017
  - întrebați, întrebați, întrebați
  - folosiți forumul
  - ajutați colegii pe forum
  - `https://ocw.cs.pub.ro/courses/uso/regulament`
    - secțiunea ,,Recomandări de comunicare electronică”
    - secțiunea ,,Realizarea temelor”
- slide-urile cursului suplimentar: ,,Sistem încorporate” vor fi publicate vineri, 1 decembrie 2017
- laboratorul 10: ,,Shell scripting” începe luni, 4 decembrie 2017

- reținerea anumitor valori/informații (în variabile)
- afișarea caracterelor speciale
- operații pe mai multe intrări dintr-o dată

Inițializare variabile în shell

```
ana@host:~$ a=10
ana@host:~$ b=ana
ana@host:~$ c="ana are mere"
ana@host:~$ d="$b $a"
ana@host:~$ echo $a, $b, $c, $d
10, ana, ana are mere, ana 10
```

- **nu** se folosește spațiu înainte și după egal la inițializare
  - s-ar considera comandă cu argumente
- folosirea valorii unei variabile se realizează prin prefixarea cu simbolul $ (*dollar*)
- variabilele shell **nu** au tip (întreg, șir, pointer)

Variabile speciale shell

- documentație: `man bash`, și căutare după *Special Parameters*
- `$?` – valoarea de retur a ultimei comenzi
- `$$` – PID-ul procesului shell curent
- `$!` – PID-ul ultimului proces (job) lansat în background
- `$_` – ultimul argument al ultimei comenzi

- definesc contextul de rulare a unui proces
- sunt moștenite de procesele copil
- exemple: `PATH`, `HOME`, `PWD`, `USERNAME`, `SHELL`
- operații specifice cu variabile de mediu
  - listare: `env`, `printenv`
  - configurare variabilă ca variabilă de mediu (exportare)
    - `export JAVA_HOME`
    - `export JAVA_HOME=/home/student/sun-java-jdk7 # init and export`
  - persistența configurării (*startup configuration*)
    - în fișierul `~/.bashrc`

Exemple de escaping

```
ana@host:/tmp/uso$ ls -l
total 0
-rw-r--r-- 1 ana ana 0 2007-11-18 00:14 alfa beta.txt
-rw-r--r-- 1 ana ana 0 2007-11-18 00:14 d.txt
ana@host:/tmp/uso$ ls alfa beta.txt
ls: alfa: No such file or directory
ls: beta.txt: No such file or directory
ana@host:/tmp/uso$ ls "alfa beta.txt"
alfa beta.txt
ana@host:/tmp/uso$ ls 'alfa beta.txt'
alfa beta.txt
ana@host:/tmp/uso$ ls alfa\ beta.txt
alfa beta.txt
ana@host:/tmp/uso$ echo $a
4
ana@host:/tmp/uso$ echo "$a"
4
ana@host:/tmp/uso$ echo \$a
$a
ana@host:/tmp/uso$ echo '$a'
$a
```

Citare (escaping) (cont.)

- escaping folosind \ (*backslash*)
  - forțează forma literală a caracterului următor
  - rol special în cazul comenzii echo -e

```
ana@host:~$ echo -e "a\tb\tc"
a       b       c
```

- escaping folosind " (ghilimele, *quote*)
  - escapează textul dintre ghilimele
  - nu escapează expandarea

```
ana@host:~$ echo "* ; && | > $sir"
* ; && | > file.txt
```

- ’ (apostrof, *single quote*)
  - escapează orice între apostroafe

```
ana@host:~$ echo '\\\$a$bc$de'
\\\$a$bc$de
```

Expandarea în shell

Expandarea variabilelor

```
ana@host:-$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/usr/local/PacketTracer5
ana@host:-$ echo $PATH | tr ':' '\n'
/usr/local/bin
/usr/bin
/bin
/usr/local/games
/usr/games
/usr/local/PacketTracer5
```

Expandarea comenzilor

```
ana@host:~$ ls
ana@host:~/tmp$ touch $(seq -f "file-%02g.txt" 1 20)
ana@host:~/tmp$ touch $(seq -f "file-%02g.txt" 1 20)
ana@host:~$ ls
file-01.txt  file-03.txt  file-05.txt  file-07.txt  file-09.txt  file-11.txt  file-13.txt  [...]
file-02.txt  file-04.txt  file-06.txt  file-08.txt  file-10.txt  file-12.txt  file-14.txt  [...]
```

Expandarea aritmetică

```
ana@host:~$ a=3
ana@host:~$ ((a++))
ana@host:~$ b=$((a+1))
ana@host:~$ echo $b
5
```

Folosire globbing în comenzi shell

```
ana@host:~/code/tests$ ls
Makefile  endian.c  struct_bit    struct_bit.o  tcp_new_sock.c
endian    endian.o  struct_bit.c  tcp_new_sock  tcp_new_sock.o
ana@host:~/code/tests$ ls *.c
endian.c  struct_bit.c  tcp_new_sock.c
ana@host:~/code/tests$ ls end?an.c
endian.c
ana@host:~/code/tests$ ls [a-s]*.c
endian.c  struct_bit.c
ana@host:~/code/tests$ ls *.{c,o}
endian.c  endian.o  struct_bit.c  struct_bit.o  tcp_new_sock.c  tcp_new_sock.o
```

Globbing (cont.)

- pentru a rula o comandă pe mai multe fișiere
- filename expansion (și altele)
- `*` – orice caracter de oricâte ori
- `?` – orice caracter
- `[abcde]`, `[a-e]` – set de caractere
- `[^abc]` – diferit de setul de caractere
- `{jpg,png,bmp}` – selecție între variante

One liner

- înlănțuirea unor comenzi
- de obicei legate prin operatorul *pipe* (|)
- `http://www.bashoneliners.com/`

Moduri de input pentru o comandă

- o comandă primește parametri
  - `touch file1 file2 ...`
  - `zip -r my.zip path1 path2 ...`
  - `gcc -o exec obj1 obj2 ...`
- folosite în conjuncție cu operatorul de *command epxansion*: `$(...)`
  - `zip -r $(generic_command that outputs file names)`
- o comandă primește input la intrarea standard (filtre de text)
  - `gemeric_command | grep 'include'`
  - `generic_command | sort -t ':'  -k 2`
  - `generic_command | sed 's/ana/bogdan/g'`

- *All is text.* (Unix)
- comenzi care ,,filtrează" text

Sintaxă pentru filtre de text

```
filtru < input_file > output_file
command1 | filtru | command2
```

- operații
  - selectare linii
  - selectare coloane (după separator)
  - prelucrare linii (modificare elemente)
  - contabilizare
- `cat`, `tac`, `rev`, `nl`, `head`, `tail`, `sort`, `grep`, `column`, `paste`, `join`, `sed`, `awk`

Ce este un script shell?

- o înșiruire de comenzi shell
- este interpretat de shell: se interpretează, pe rând, fiecare comandă
- pe lângă comenzi externe, sunt folosite și comenzi interne precum `if`, `for`, `while`, `case`
- facilități shell precum *command expansion*
- variabile

Variabile

- `server=141.85.224.218`
- `ssh -l root "$server"`
- `username="johnny"`
- `sudo useradd -m -d /home/"$username" -s /bin/bash "$username"`
- `base_folder="/home/student/uso/scripts"`
- `zip -r scripts.zip "$base_folder"`

- pentru one linere folosim, în general, filtre de text
- pentru lucruri mai ample folosim `while read`; pentru lucruri mai complexe `awk` și `sed`
- `IFS`: *Input Field Separator*
- `while + read`: sunt split câmpurile fiecărei linii și trecute în variabile
- variabilele pot fi apoi folosite în shell

Construcția while read

Extragerea numelui de utilizator și a directorului home

```
IFS=":"; while read user x y z t home v; do echo "$user:$home";
         done < /etc/passwd
```

Selectarea după o notă

```
IFS=$'t'; while read student group grade; do if test grade -ge 7; then
          echo "$student,$grade"; fi; done < students.txt
```

Parcurgerea fișierelor dintr-un director

```
for f in *; do
    cat "$f"
done
```

Parcurgerea elementelor dintr-o listă

```
for group in "311CA" "312CA" "313CA" "314CA" "315CA"; do
    no=(./get-students-in-group "$group")  # custom script
    echo "Group $group has $no students."
done
```

Parcurgerea utilizatorilor din sistem

```
for user in $(cut -d ':' -f 1); do
    nlogins=$(last "$user" | grep "^$user" | wc -l)
    echo "User $user logged in $nlogins times recently."
done
```

Check if file exists

```
if test -f "$file"; then
    echo "File $file exists."
fi
```

Check if number is between 1 and 5

```
if test "$num" -ge 1 -a "$num" -le 5; then
    echo "$num is between 1 and 5."
else
    echo "$num is not between 1 and 5."
fi
```

Check if proper number of arguments

```
if test "$#" -ne 1; then
    echo "Usage: $0 argument"
    echo "You must provide a single argument to the script."
    exit 1
fi
```

Utilizare

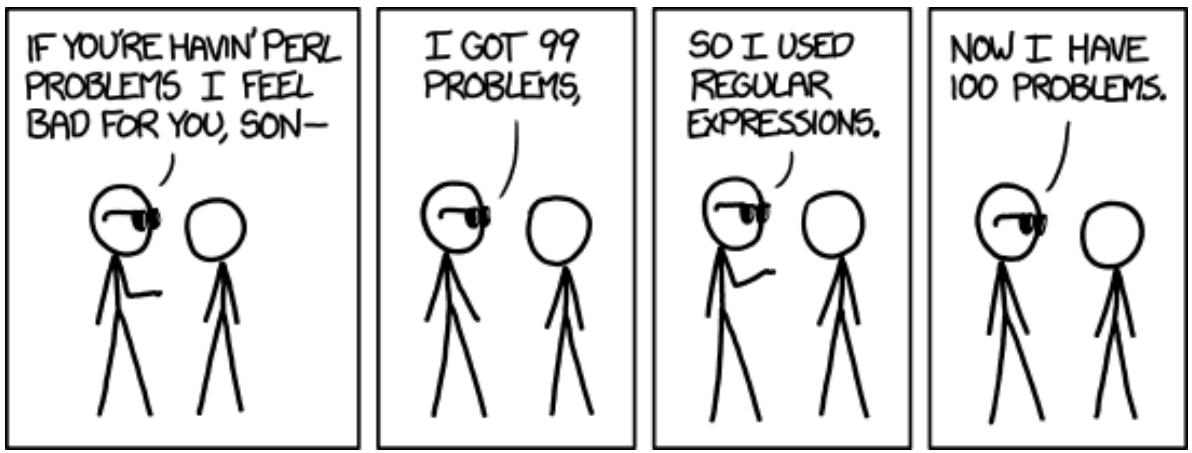
- *regular expressions* sau *regex*
- căutare (*searching*, *pattern matching*)
- validarea unui șir, a unui text, a unei intrări (număr de telefon, URL, nume de variabilă, adresă MAC)
- substituirea unei expresii

Exemple grep

Check if file exists

```
grep '#include'
grep '^include'
grep '^[ ]*include'
grep '^[ \t]*$'
```

Probleme cu expresii regulate



http://xkcd.com/1171/

http://www.twoproblems.com/

Când să **nu** folosești expresii regulate

- când există parsere
- când există aplicații, tool-uri, funcții mai bune pentru a face acest lucru
- pentru anumite tipuri de date (adrese de e-mail, HTML): fie nu se poate, fie e foarte greu, fie va ieși o "arătare" de cod
- atunci când, folosite abuziv, fac codul nementenabil/nelizibil

Exemple de expresii regulate

Expresie regulată	**Explicație**
`[_a-zA-Z][_0-9a-zA-Z]*`	nume de variabilă/funcție
`07[:digit:]{8}`	număr de telefon
`[:upper:][:alpha:]+`	numele unei persoane
`[:digit:]{2}\.(pdf|png|svg)`	nume fișiere din două cifre și diferite extensii
`([01][:digit:]|2[0-3]):[0-5][:digit:]`	timp în ore și minute
`^[:upper:]{1,10}$`	linii conținând maxim 10 majuscule

Metacaractere în expresii regulate

Metacaracter	**Efect**
^ (*caret*)	început de linie
$ (*dollar*)	sfârșit de linie
. (*dot*)	orice caracter
[...] (*square brackets*)	set de caractere
? (*question mark*)	expresia anterioară cel mult o dată
* (*star*)	expresia anterioară de oricâte ori posibil niciodată
+ (*plus*)	expresia anterioară de oricâte ori cel puțin o dată
e1|e2 (*or*)	expresia de dinainte **sau** cea de după

regex vs. globbing

- globbing este folosit în shell, în special pentru *filename expansion*
- globbing poate fi considerat o formă mai slabă de expresie regulată
- expresiile regulate sunt puternice, cu mai multe cazuri de utilizare și suport în majoritatea limbajelor/framework-urilor moderne

Metacaracter	**Rol în globbing**	**Rol în regex**
.	nimic special	orice caracter
[...]	set de caractere	set de caractere
*	orice caracter de oricâte ori	expresia anterioară de oricâte ori
?	orice caracter	expresia anterioară o dată sau niciodată
~	directorul home	nimic special

Suport pentru expresii regulate

- utilitare shell: grep, awk, sed
- limbaje de programare: Perl, Python, PHP, Ruby, Java, JavaScript, C++11
- editoare: Vim, Emacs
- baze de date
- biblioteci cu suport de expresii regulate: Gnulib

- prelucrare de date
  - task-uri simple
  - *quick'n'dirty*
  - se folosesc filtre de text: prelucrează text (la intrare, și la ieșire): `head`, `tail`, `grep`, `cut`, `tr`, `awk`, `sed`
- automatizare
  - task-uri repetitive
  - se pun comenzile în cauză într-un script shell
  - se rulează scriptul la nevoie

- mai multe comenzi (eventual înlănțuite)
- e vorba de comenzi comune, nu neapărat (chiar foarte rar) filtre de text
  - poate fi vorba de comenzi de prelucrare de imagini
  - poate fi vorba de comenzi de procesare de fișiere (din LaTeX în PDF)
  - poate fi vorba de descărcat fișiere de la distanță
- scriptul se rulează periodic sau la nevoie
- nu este obligatoriu să folosiți construcții precum `for`, `while` sau `if`
- un script trebuie să își facă treabă, nu trebuie să fie limbaj de programare
- puteți avea one linere în script

Mindset legat de shell scripting

- să fie *quick'n'dirty*; dacă e prea complicat trebuie mers la un limbaj de programare
- *keep it simple*: nu folosi expresii regulate sau construcții complicate fără să fie cazul; a.k.a. nu te lega la cap fără să te doară
- mintea la obiectiv: ce vreau să obțin?
- *best tool for the best job*
- folosește tot ce există, nu reinventa roata, *DRY: don't repeat yourself*
- documentează-te în prealabil: măsoară de două ori si taie o dată
- este posibil să existe o comandă sau opțiunea unei comenzi care să facă (parte din) ceea ce dorești
- niciodată nu vei ști suficient de mult shell scripting: documentează-te, exersează și îmbunătățește

Sfaturi legate de shell scripting

- nu folositi shell scripting pentru ceea ce se face mai bine/usor/eficient in Python, C, Java, Ruby, PHP etc.
- nu reinventați roata
- folosiți ghilimele când referiți valoarea unei variabile
- folosiți cea mai bună opțiune (`cut`, `grep`, `tr`, `awk`, `sed`, `while read`) după nevoie
- folosiți opțiuni de tip expresii regulate sau similare între apostrofuri
- `http://mywiki.wooledge.org/BashFAQ`

- două cazuri de utilizare: prelucrare de date și automatizare de date
- două tipuri de input pentru comenzi: parametri și intrarea standard (filtre de text)
- *keep it simple*
- *don't repeat yourself*
- expresille regulate pot să încurce în loc să ajute
- nu este obligatoriu să folosiți `for`, `while`, `if`; shell scripting nu e programare

Cuvinte cheie

- variabile
- variabile de mediu
- escaping
- expandare
- one liner
- filtru de text
- script shell
- `IFS`
- `while read`
- `for`
- `if`
- expresii regulate
- metacaractere
- globbing
- `grep`
- prelucrare de date
- automatizare

Resurse utile

- Jon Goyvaerts, Steven Levithan: Regular Expressions Cookbook
- `http://www.regular-expressions.info/`
- `http://regexpal.com/`
- `http://www.regexr.com/`
- `http://www.twoproblems.com/`
- `http://koala.cs.pub.ro/training/wiki/medium-unix/home`
- `http://koala.cs.pub.ro/training/wiki/advanced-unix/home`

- Jan Goyvaerts, Steven Levithan
- creatorii `http://www.regular-expressions.info/`, `http://regexpal.com/`
- 2nd Edition
- practică, multe ,,rețete”
- o recenzie aici: `http://blog.codinghorror.com/regular-expressions-for-regular-programmers/`

Larry Wall



http://en.wikipedia.org/wiki/File:Larry_Wall_YAPC_2007.jpg

- creatorul Perl
- BDFL pentru proiectul Perl
- autorul programului `patch`
- câștigător al *International Obfuscated C Code Contest*

Rackspace

- companie de hosting IT
- fondată în 1996
- unul dintre principalii furnizori de servicii cloud (`Rackspace Cloud`)
- contribuitor la OpenStack
- folosit de GitHub

Lua

- limbaj de programare de scripting
- paradigme multiple (la fel ca Python)
- proiectat pentru a fi ușor încorporat în alte limbaje
- API C simplu
- limbaj de scripting pentru dezvoltatorii de jocuri
- folosit de limbajul de scripting al utilitarului `nmap`
- folosit pentru configurarea managerului de ferestre Awesome